

PostgreSQL in the Age of AI

From a single SQL query to a production data system and an AI answer.

POSTGRES^{SQL}



Technical detail

PostgreSQL provides transactions, extensible data types, indexes and an execution engine. AI retrieval is one workload on that foundation.

```
SELECT answer FROM knowledge WHERE question = $1;
```

When should you choose PostgreSQL?

Choose it when relationships, constraints, SQL and reliable transactions matter; measure when a specialized store is a better fit.

A GOOD DEFAULT FOR APPLICATION DATA

- 01 **Related records**
- 02 **Constraints + ACID**
- 03 **SQL + indexes**
- 04 **Extensions**

Compare your workload: PostgreSQL fits transactional apps with relationships. A cache or analytics warehouse can serve other jobs.

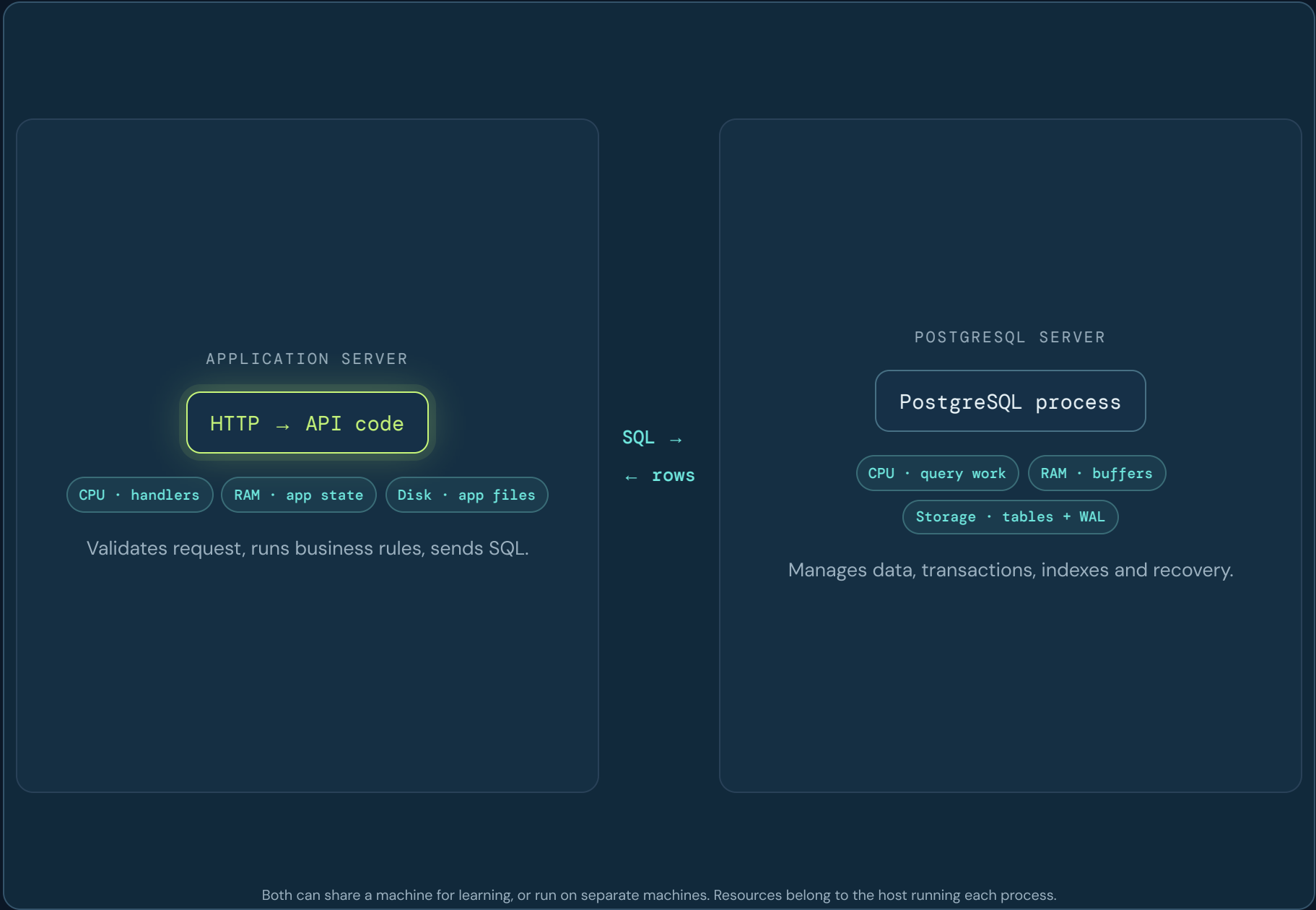
Technical detail

PostgreSQL is a strong default for transactional applications and mixed relational/JSON workloads. A cache such as Redis suits ephemeral key-value speed; a columnar warehouse suits large scans and analytics. Dedicated search or vector systems may fit specialized scale. PostgreSQL extensions can broaden its range, but CPU, RAM, I/O and operations still set limits.

```
CREATE TABLE users (id bigint PRIMARY KEY, email text UNIQUE);
```

Your API server is not your database

The API runs application code. PostgreSQL is a separate server process with its own CPU, RAM, storage and connections.



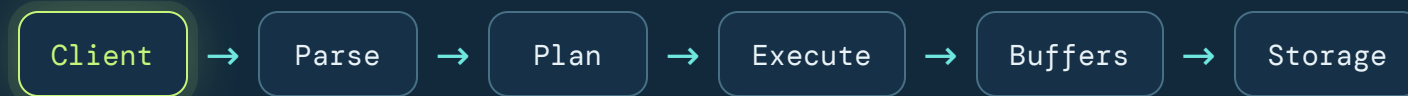
Technical detail

PostgreSQL is open source and may run on your own machine, VM, container or managed service. The DB server plans queries, manages shared buffers, WAL, files and background workers; the API server handles HTTP, business logic and external calls.

-- API sends SQL over a connection to a PostgreSQL server

PostgreSQL in one picture

A request crosses connection handling, parsing, planning, execution, memory and storage.



Technical detail

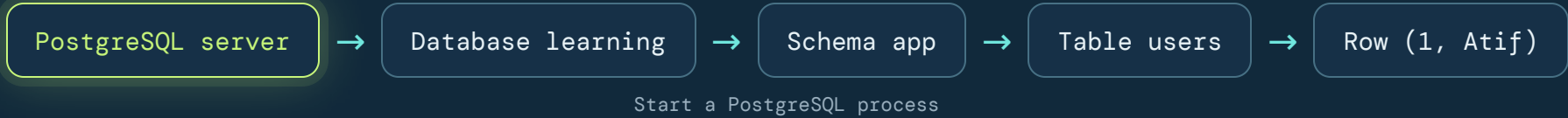
Parser validates syntax and creates a query tree; planner chooses an access plan; executor runs it. Shared buffers cache pages. WAL records changes for durability.

```
SELECT * FROM users WHERE email = $1;
```

From PostgreSQL server to your first row

Start the server, create a database, connect, create a schema and table, then insert and query a row.

BUILD ONE LAYER AT A TIME



Technical detail

A PostgreSQL instance manages multiple databases. CREATE DATABASE requires CREATEDB or superuser privilege and cannot run inside a transaction block. Schemas group objects inside one database; tables have typed columns and constraints. The server process must be running before psql can connect.

```
CREATE DATABASE learning;
-- reconnect to learning
CREATE SCHEMA app;
CREATE TABLE app.users (id bigint PRIMARY KEY, name text NOT NULL);
INSERT INTO app.users VALUES (1, 'Atif');
SELECT * FROM app.users;
```

Database → schema → table → row → column

A column is one named, typed field; a row (tuple) is one record; a table groups rows of the same shape.

SERVER / LEARNING DATABASE / APP SCHEMA / USERS TABLE

id bigint	name text	email text
1	Atif	atif@example.com
2	Mira	mira@example.com
3	Sam	sam@example.com

Table = named collection of similarly shaped rows.

Technical detail

A PostgreSQL cluster can contain databases; databases contain schemas; schemas contain relations. Types and constraints encode invariants.

```
CREATE TABLE users (id bigint PRIMARY KEY, name text NOT NULL, email text UNIQUE);
```

A relation is a mathematical idea, not an FK line

In the relational model, a relation is a set of tuples with named attributes. A SQL table is its practical counterpart.

RELATION • MATH IDEA

{ (id, name) → (1, Asha), (2, Atif) }

Tuple = one row • attribute = one column

agencies

id = 7

name = North

← agency_id = 7

donors

id = 42

agency_id = 7

A relation describes rows with named columns.

Technical detail

Tuple is the relational-model word for row; attribute corresponds to column. SQL permits duplicate rows and NULL, so real SQL tables are not perfect mathematical relations. A foreign key is a separate constraint linking values between tables. It enforces validity; it does not itself store a pointer, join the tables, or automatically index the referencing column.

```
CREATE TABLE agencies (id bigint PRIMARY KEY, name text);
CREATE TABLE donors (id bigint PRIMARY KEY, agency_id bigint
REFERENCES agencies(id));
```

Relationships make data useful

A primary key identifies a row. A foreign key checks that a value refers to an allowed row in another table.



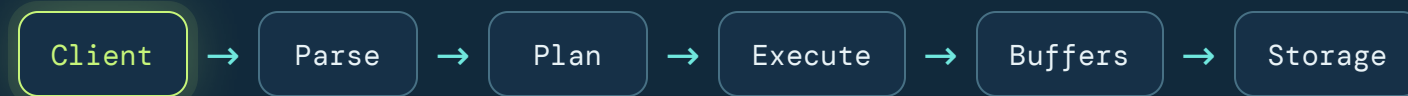
Technical detail

One-to-many uses an FK on the many side. Many-to-many uses a bridge table with two FKs. One-to-one uses a unique FK.

```
CREATE TABLE donors (id bigint PRIMARY KEY, agency_id bigint REFERENCES agencies(id));
```


What happens when we run SQL?

PostgreSQL turns text into a plan, then the executor visits the needed pages and returns rows.



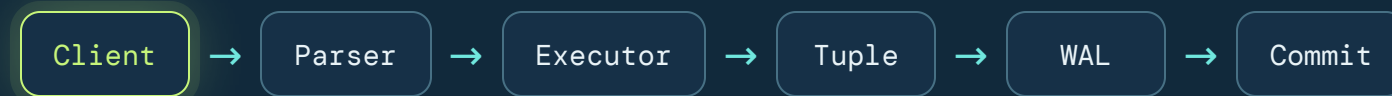
Technical detail

Planner estimates cardinalities from statistics. Execution may scan sequentially, use an index, sort, join or aggregate.

```
SELECT id, name FROM users WHERE email = 'atif@example.com';
```

What happens during INSERT?

A write creates a new tuple and WAL records the change before commit is acknowledged as durable.



Technical detail

WAL flush and data-page flush are different events. A dirty buffer may reach the data file later; recovery can replay durable WAL.

```
INSERT INTO users(name,email) VALUES ('Atif','atif@example.com');
```

A transaction keeps changes together

Move ₹2,000 between accounts. Either both updates commit or neither does.



Technical detail

Atomicity, consistency, isolation and durability describe different guarantees. This demo illustrates atomicity; correctness also needs constraints and concurrency handling.

```
BEGIN;  
UPDATE accounts SET balance=balance-2000 WHERE id=1;  
UPDATE accounts SET balance=balance+2000 WHERE id=2;  
COMMIT;
```

Who reads, who waits, and who sees the new version?

T1 updates a row and holds its row lock. T2 tries to update that same row and waits. A normal reader can still see a suitable committed version.

ILLUSTRATIVE ORDER · NO CLAIMED DATABASE TIMING

T1 · writer

idle

T2 · writer

idle

Reader

idle

v1 · committed

v2 · pending

v3 · pending

Initial committed row v1 = ₹10,000

No universal transaction time: lock wait lasts until T1 ends or a timeout occurs. Measure elapsed time and lock waits in your workload; keep transactions short.

Technical detail

Under READ COMMITTED, a statement sees a snapshot established when it begins. UPDATE creates a new tuple version and takes a row-level lock. A concurrent UPDATE of that row must wait for the first transaction, then recheck the row after the lock is released. Plain SELECT normally reads a visible committed version without waiting on that row lock. Actual transaction duration and wait time depend on application behavior, locks, I/O and load; there is no universal number.

```
-- Session A
BEGIN; UPDATE accounts SET balance=8000 WHERE id=1;
-- Session B: UPDATE accounts SET balance=9000 WHERE id=1; --
waits
-- Session C: SELECT balance FROM accounts WHERE id=1; -- sees
committed version
COMMIT;
```

MVCC: readers see a snapshot

An update creates a new row version. Existing readers can continue using a version visible to their snapshot.



Simplified READ COMMITTED snapshot sequence: an already-running statement sees its snapshot; a later statement may see the committed new row.

Technical detail

Visibility uses transaction IDs and snapshots. UPDATE creates a new tuple version; locks still exist for conflicting writes. Old versions become removable only when no relevant snapshot needs them.

```
BEGIN;  
SELECT balance FROM accounts WHERE id=1;  
-- another transaction updates the row
```

The same query gets expensive

As row count grows, a full table scan examines more data even when the result stays one row.

10 rows



One matching email · more candidates to inspect in a full scan

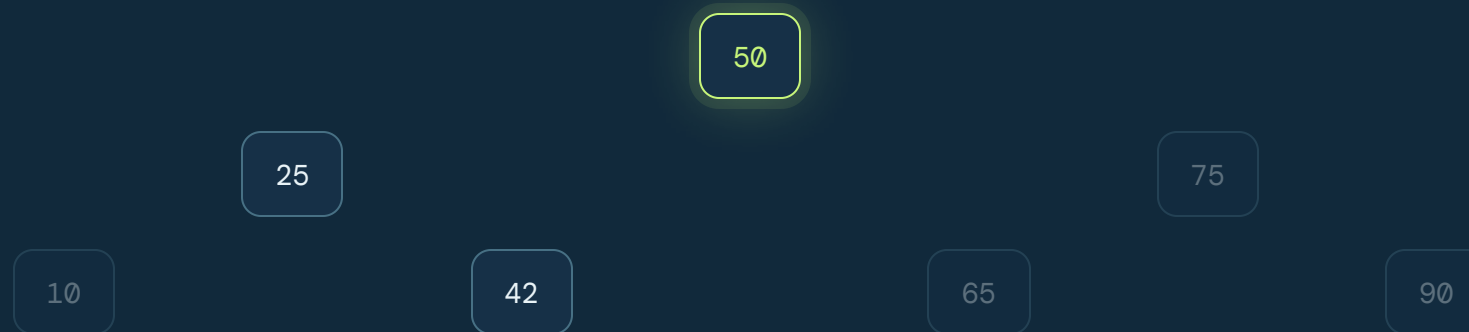
Technical detail

Selectivity, table size, cache state, statistics and row width all affect plan choice and elapsed time.

```
SELECT * FROM demo_users WHERE email = 'user500000@example.test';
```

An index narrows the search

A B-tree is another structure that helps locate candidate rows; it costs space and write work.



Find 42: inspect a path, then fetch the row

Technical detail

B-tree supports equality and ranges. A matching index is not automatically used for low-selectivity predicates or tiny tables. Composite and partial indexes tailor access paths.

```
CREATE INDEX demo_users_email_idx ON demo_users(email);
```

EXPLAIN ANALYZE: PostgreSQL's receipt

Run the query, inspect its plan tree, then compare estimated rows with actual rows, time, loops and buffers.

PREPARED · STRUCTURE ONLY

EXPLAIN (ANALYZE, BUFFERS) SELECT ... WHERE email = ?

Seq Scan demo_users

Estimated rows	1	Actual rows / loop	—
Planner cost (units)	—	Actual time (ms)	—
Loops	—	Filtered out	—
Buffer hits	—	Buffer reads	—

Costs are estimates in planner units. Actual time appears only after running the query. Rows are emitted per loop, not all rows examined.

Click a node to read estimated rows, actual rows, loops, time and buffers.

Technical detail

Costs are planner units, not milliseconds. `rows` is output per loop; `Rows Removed by Filter` is useful for understanding scan work. Timings depend on hardware and cache.

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)
SELECT * FROM demo_users WHERE email = 'user500000@example.test';
```


Make it slow. Measure. Fix it.

Prepare a real dataset, run the exact query, add one index, and compare actual PostgreSQL plans.

PERFORMANCE LAB · PREPARED TEACHING MODE

1,000,000 rows

Illustrative plan only. No measured timing is shown.

Technical detail

The local API only accepts fixed benchmark operations. 100K/1M are fast mode; 5M/10M require full mode. A prepared view never invents execution time.

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)
SELECT * FROM demo_users WHERE email = 'user500000@example.test';
```

Where does a row live?

Relations are made of pages; pages hold tuple versions. Shared buffers cache pages in memory.

ILLUSTRATIVE 8 KB PAGE



Relation → page → tuple version; shared buffers cache pages

Technical detail

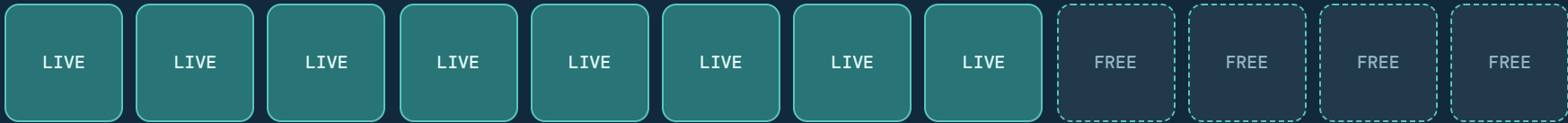
Most PostgreSQL builds use 8 KB pages. TOAST stores oversized values out of line. Visibility metadata and line pointers complicate this simplified drawing.

```
SELECT pg_size_pretty(pg_relation_size('demo_users'));
```

Why VACUUM exists

UPDATE and DELETE leave obsolete versions. VACUUM reclaims eligible space for reuse.

ILLUSTRATIVE PAGE · 0 OBSOLETE · 0 REUSABLE



VACUUM can make eligible dead space reusable

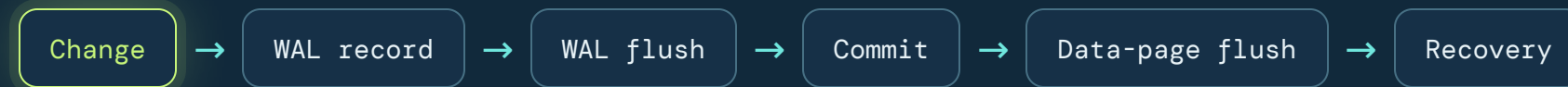
Technical detail

Autovacuum is essential for space reuse, statistics and transaction-ID wraparound prevention. VACUUM FULL rewrites a relation and takes an ACCESS EXCLUSIVE lock; it is not routine maintenance.

```
VACUUM (VERBOSE, ANALYZE) demo_users;
```

WAL lets PostgreSQL recover

Log records make committed changes recoverable even if dirty data pages were not flushed before a crash.



Technical detail

WAL records are written before corresponding dirty data pages. Checkpoints bound recovery work. WAL also feeds physical replication.

```
SHOW wal_level;
```

Primary → replicas

A primary sends WAL to replicas. Replicas can serve reads, but lag and failover need explicit handling.



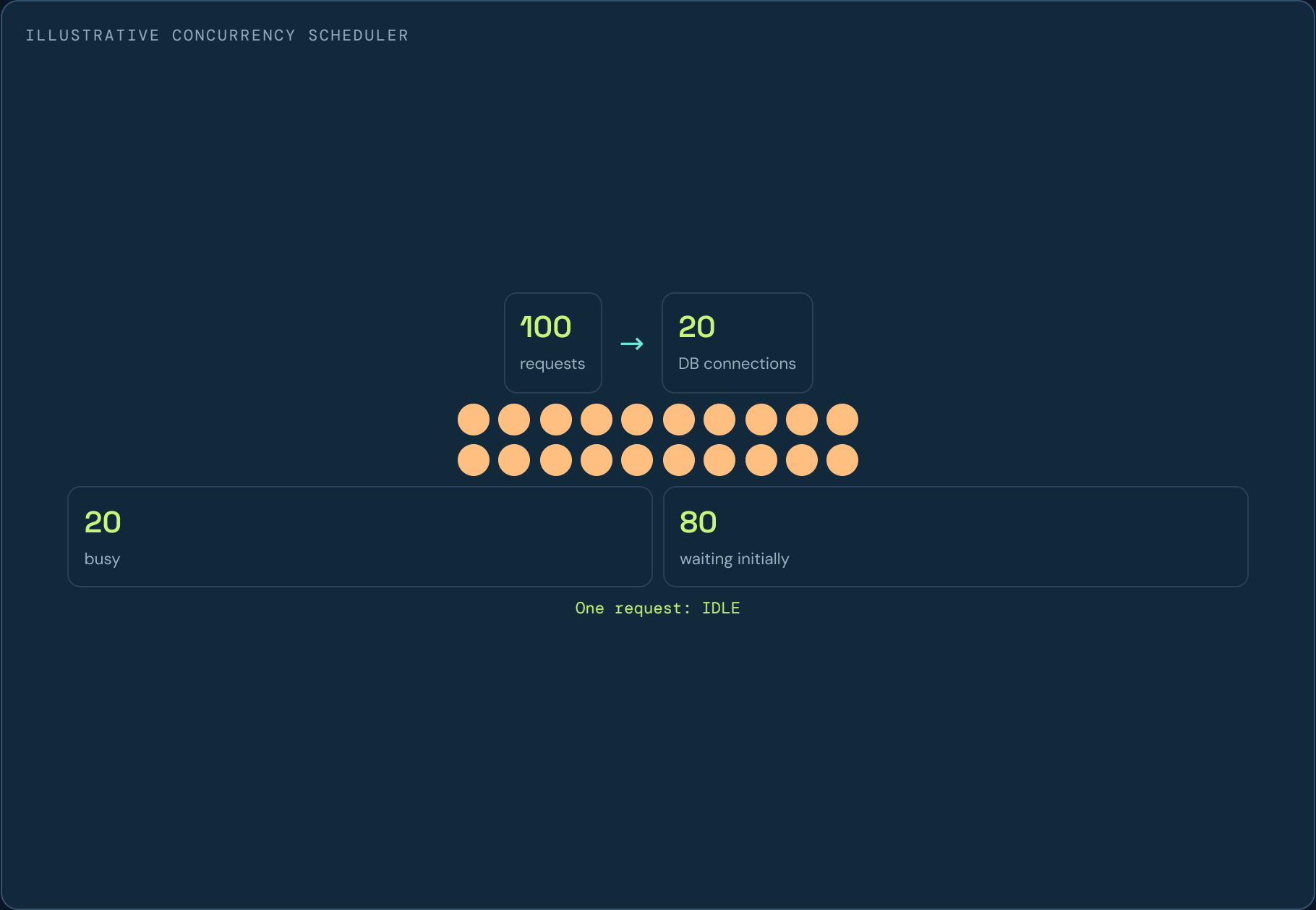
Technical detail

Streaming replication is commonly asynchronous. Read-after-write consistency cannot be assumed from a lagging replica. Replication is not a substitute for backup.

```
SELECT application_name, state, replay_lag FROM
pg_stat_replication;
```

Connection pooling controls concurrency

Requests borrow a limited set of reusable connections. Excess requests wait or time out.



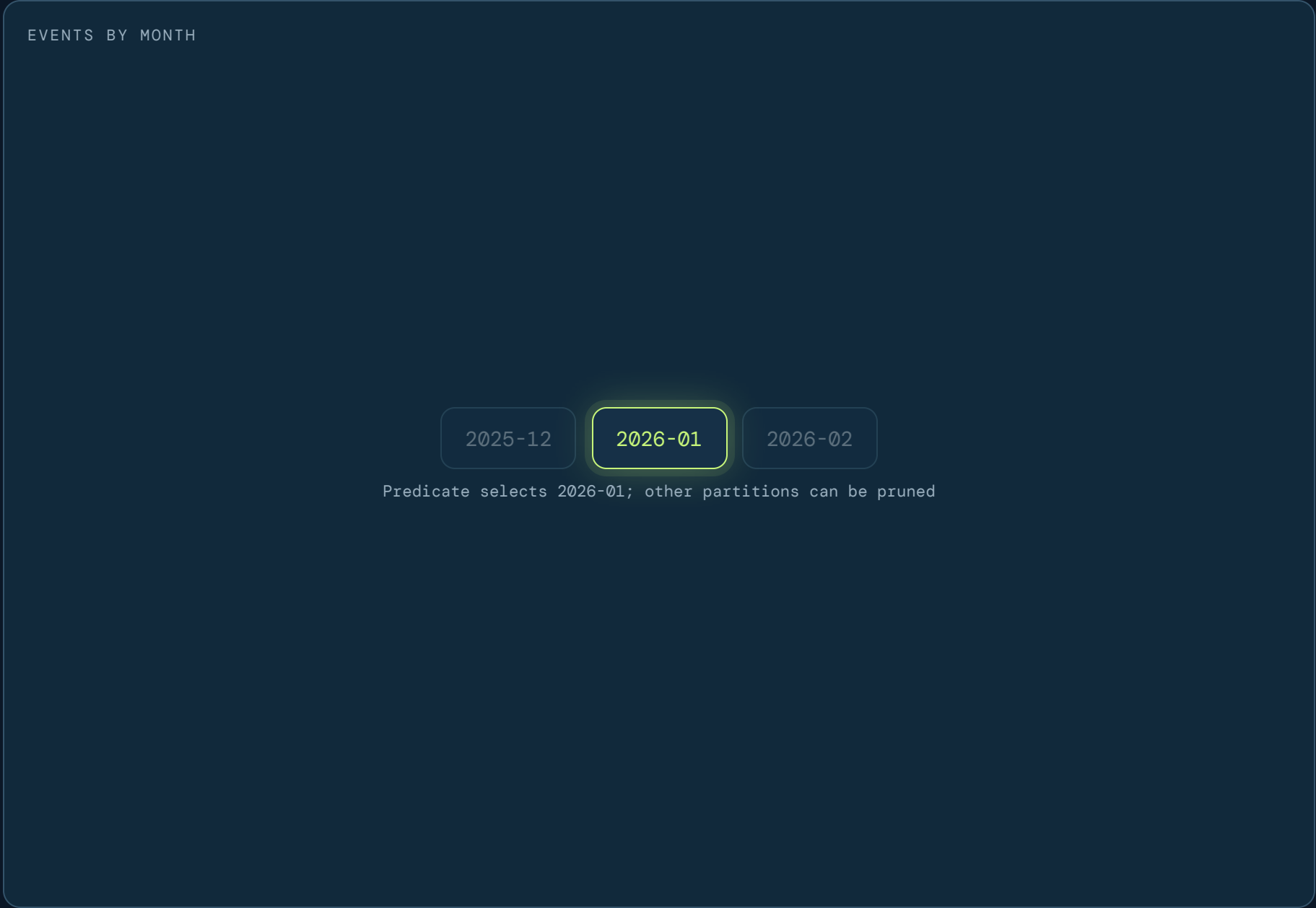
Technical detail

PostgreSQL connections consume resources. Transaction pooling changes session-state assumptions. Pool size depends on query time, CPU, memory and workload.

```
-- Configure an application pool or PgBouncer; measure wait time and DB saturation.
```

Partitioning lets PostgreSQL skip data

A date predicate can prune partitions outside the requested range.



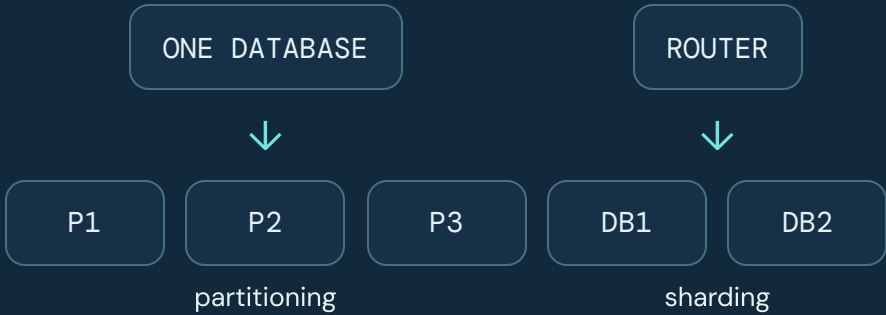
Technical detail

Partition pruning requires predicates that can be matched to partition bounds. Too many partitions and maintenance overhead can erase gains.

```
CREATE TABLE events (id bigint, created_at date) PARTITION BY RANGE (created_at);
```

Partitioning ≠ sharding

Partitions live under one logical PostgreSQL database. Shards split data across databases and add routing complexity.



Technical detail

Cache, replicas, queues and sharding address different bottlenecks. Cross-shard joins and transactions are harder. Add a component after measuring the limit it removes.

```
SELECT * FROM events WHERE created_at >= DATE '2026-01-01';
```


Self-host, serverless Postgres, or managed RDS?

Choose from workload shape, operational control, availability needs and observed CPU, RAM, I/O and connection pressure.

PICK FOR A WORKLOAD, NOT A LOGO

SELF-HOST

NEON-STYLE

RDS

Prototype or bursty app

Neon-style serverless

Autoscaling, branching and idle compute fit variable demand; watch wake latency, connection patterns and working-set memory.

CPU & query time

RAM working set

storage I/O

connections

availability

Measure representative traffic → find the saturated resource → size with headroom → load test again

Technical detail

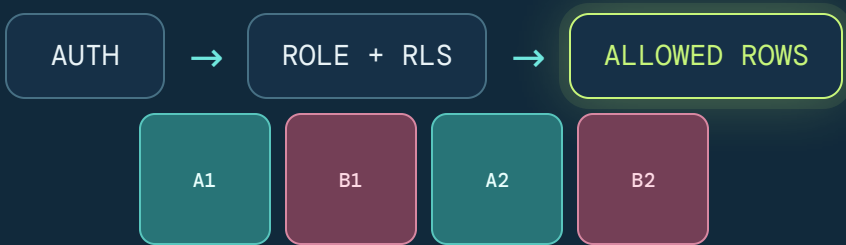
Self-host gives control and responsibility for patching, backup and failover. Neon offers separate compute/storage, autoscaling and scale-to-zero, useful for variable or idle workloads but requiring attention to wake behavior and connection patterns. RDS offers managed PostgreSQL instances with chosen compute/storage, backups and Multi-AZ options, suitable for steady or controlled workloads. None is inherently faster or cheaper for every case.

```
SELECT current_setting('server_version');
SELECT count(*) FROM pg_stat_activity;
```

Roles, grants and RLS protect rows

Authentication identifies a caller. Grants and row-level security decide what the caller may access.

TENANT A REQUEST



B rows are filtered by policy in this teaching example

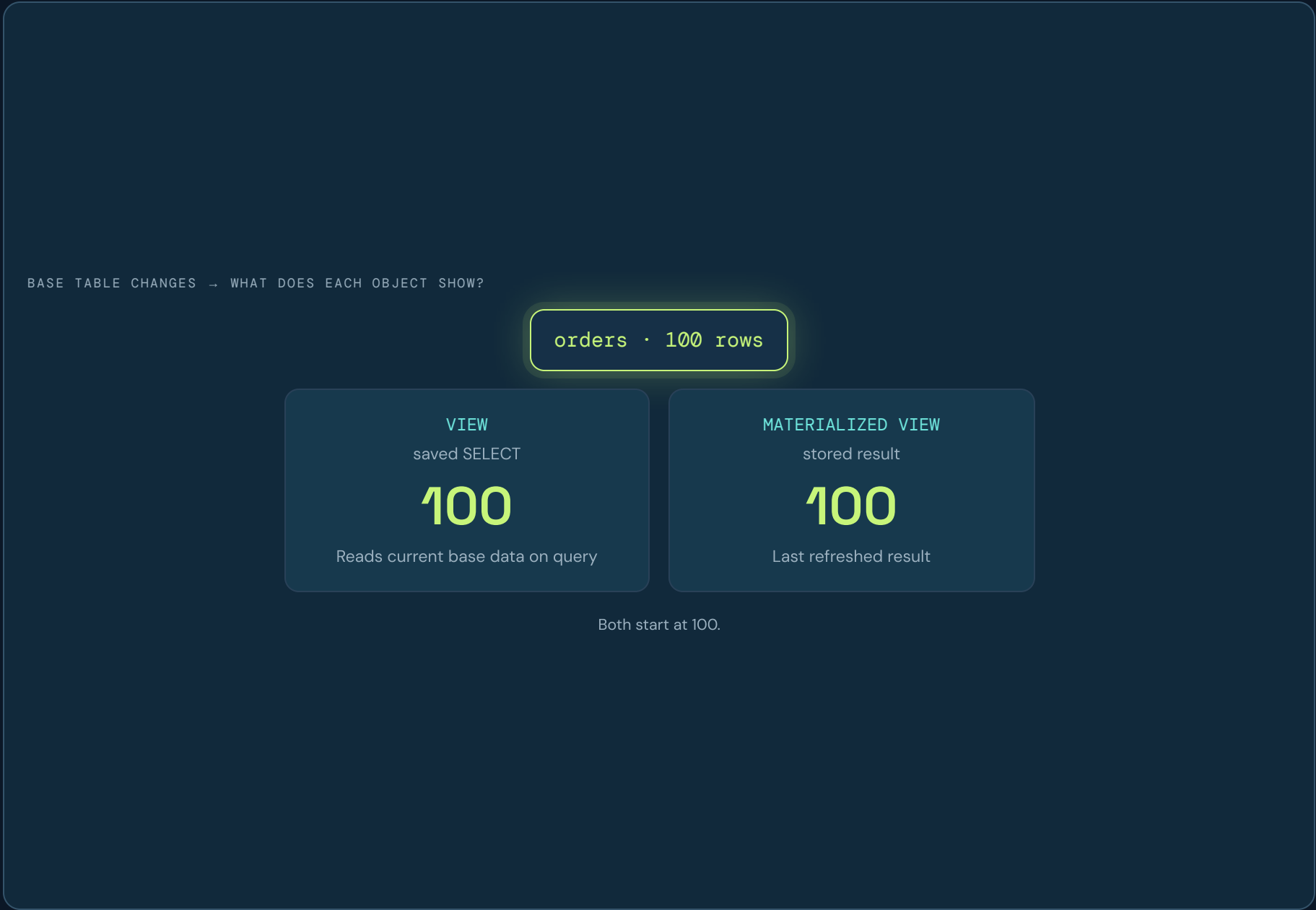
Technical detail

Role ownership and BYPASSRLS matter. Set tenant context safely per transaction; avoid trusting a client-provided tenant ID without verified identity. Test policy behavior and performance.

```
ALTER TABLE documents ENABLE ROW LEVEL SECURITY;  
CREATE POLICY tenant_read ON documents FOR SELECT USING (tenant_id  
= current_setting('app.tenant_id')::bigint);
```

Views and materialized views are different

A view stores a query definition. A materialized view stores the query result until you refresh it.



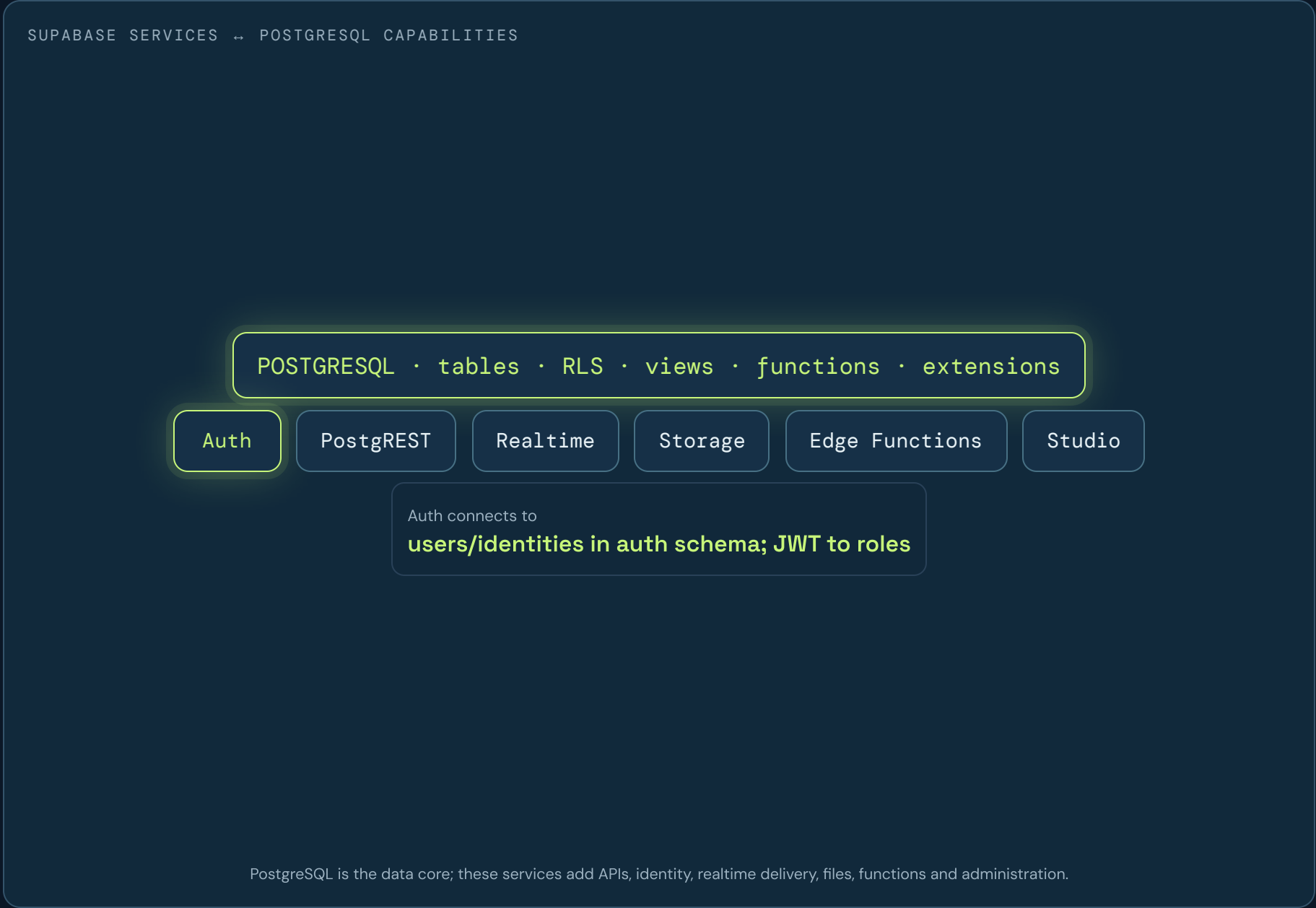
Technical detail

A normal view runs against current underlying data when queried. A materialized view persists rows and can have indexes, but its data becomes stale until refresh. REFRESH can be expensive; concurrent refresh has requirements. Permissions and RLS behavior require care, especially with owner–security views; security_invoker views can use caller permissions on supported PostgreSQL versions.

```
CREATE VIEW active_users AS SELECT id,name FROM users WHERE active;
CREATE MATERIALIZED VIEW daily_totals AS SELECT day,count(*) FROM orders GROUP BY day;
REFRESH MATERIALIZED VIEW daily_totals;
```

What Supabase builds around PostgreSQL

PostgreSQL stores the data; Auth, PostgREST APIs, Realtime, Storage, Edge Functions and Studio add product services.



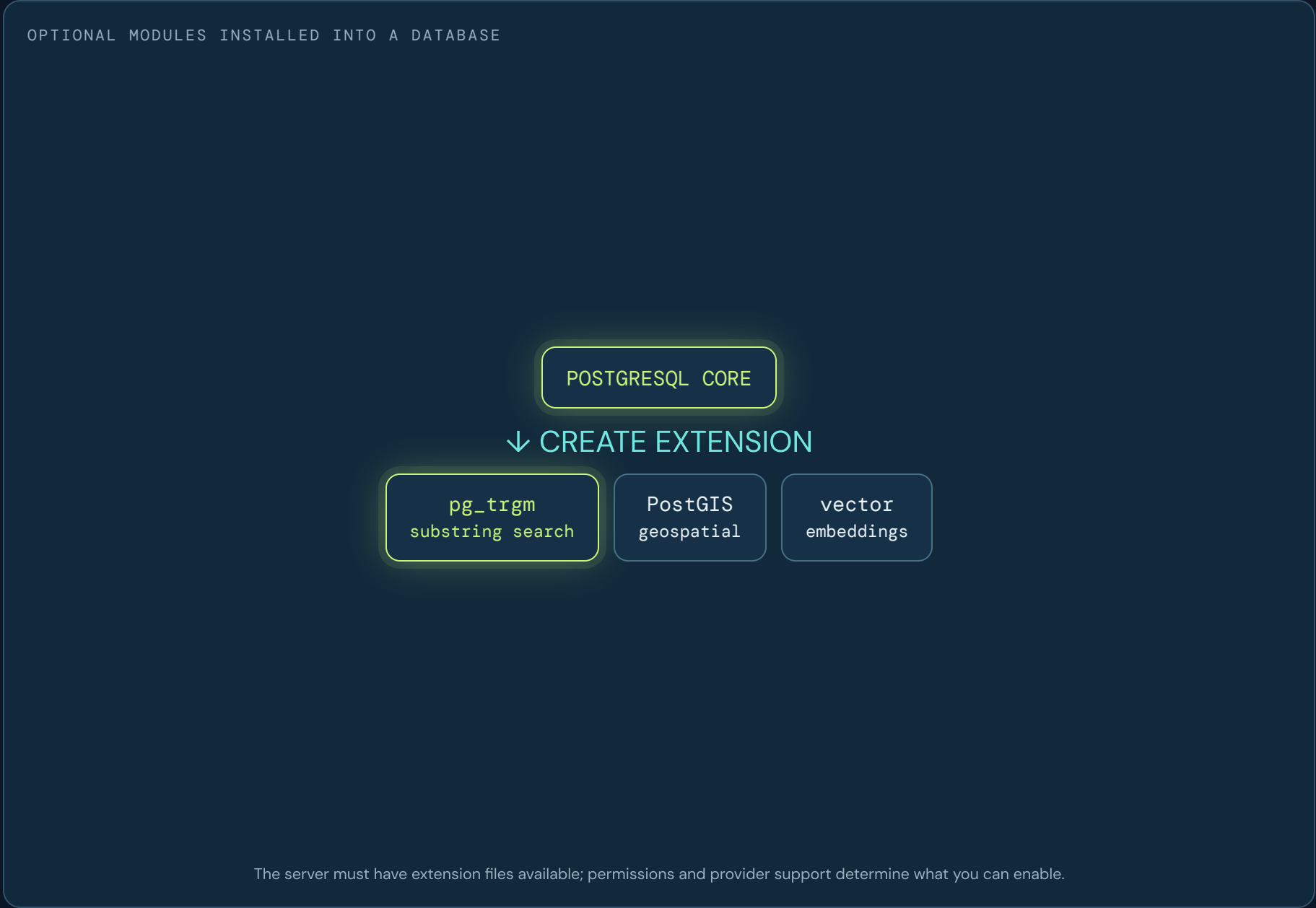
Technical detail

Supabase chose PostgreSQL as its relational, transactional and extensible data core. PostgREST exposes tables, views and functions as an API; Auth issues identity tokens, RLS can restrict rows, Realtime consumes database change streams, Storage keeps metadata in PostgreSQL while file objects live in object storage, and Edge Functions run outside the database. Studio manages objects.

```
SELECT * FROM public.documents LIMIT 3;
```

Extensions add database capabilities

Extensions install SQL objects such as types, functions, operators and index methods into a database.



Technical detail

The extension support files must exist on the server before CREATE EXTENSION can load them. Privileges vary by extension. pg_trgm adds trigram search operators/index support, PostGIS adds geospatial types/functions, and pgvector adds vector types/operators/index methods. On managed services, the provider controls what is available.

```
SELECT name,default_version FROM pg_available_extensions;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
-- pgvector is installed as extension name "vector"
CREATE EXTENSION IF NOT EXISTS vector;
```

JSONB and text search

Structured columns, JSONB, full-text search and trigram search answer different retrieval questions.



Technical detail

Use relational columns for stable keys and joins. JSONB with GIN can support containment. Leading-wildcard LIKE usually needs another strategy such as pg_trgm.

```
CREATE INDEX docs_fts_idx ON documents USING gin
(to_tsvector('english', content));
```

What is an embedding?

An embedding model maps text to a numeric vector. Nearby vectors can represent related meaning.



Illustrative 2D map; real model vectors are high-dimensional.

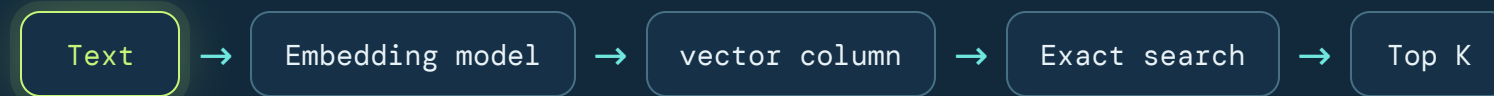
Technical detail

The displayed 2D plot is a teaching diagram, not a projection of real model vectors. Actual models commonly use hundreds or thousands of dimensions; similarity depends on model and metric.

-- Example vector is illustrative, not model output: $[0.12, -0.43, 0.77]$

pgvector adds vector search

Store vectors beside metadata and use SQL to rank nearest neighbors.



Technical detail

Exact nearest-neighbor search compares candidates. HNSW and IVFFlat are approximate indexes with speed/recall tradeoffs; test with your filters and data distribution.

```
CREATE EXTENSION IF NOT EXISTS vector;  
CREATE TABLE chunks (id bigserial PRIMARY KEY, content text,  
embedding vector(384));  
SELECT content FROM chunks ORDER BY embedding <=> $1::vector LIMIT  
3;
```


RAG retrieves evidence before answering

Question → embedding → vector search → visible chunks → context → model answer.

PREPARED EXAMPLE



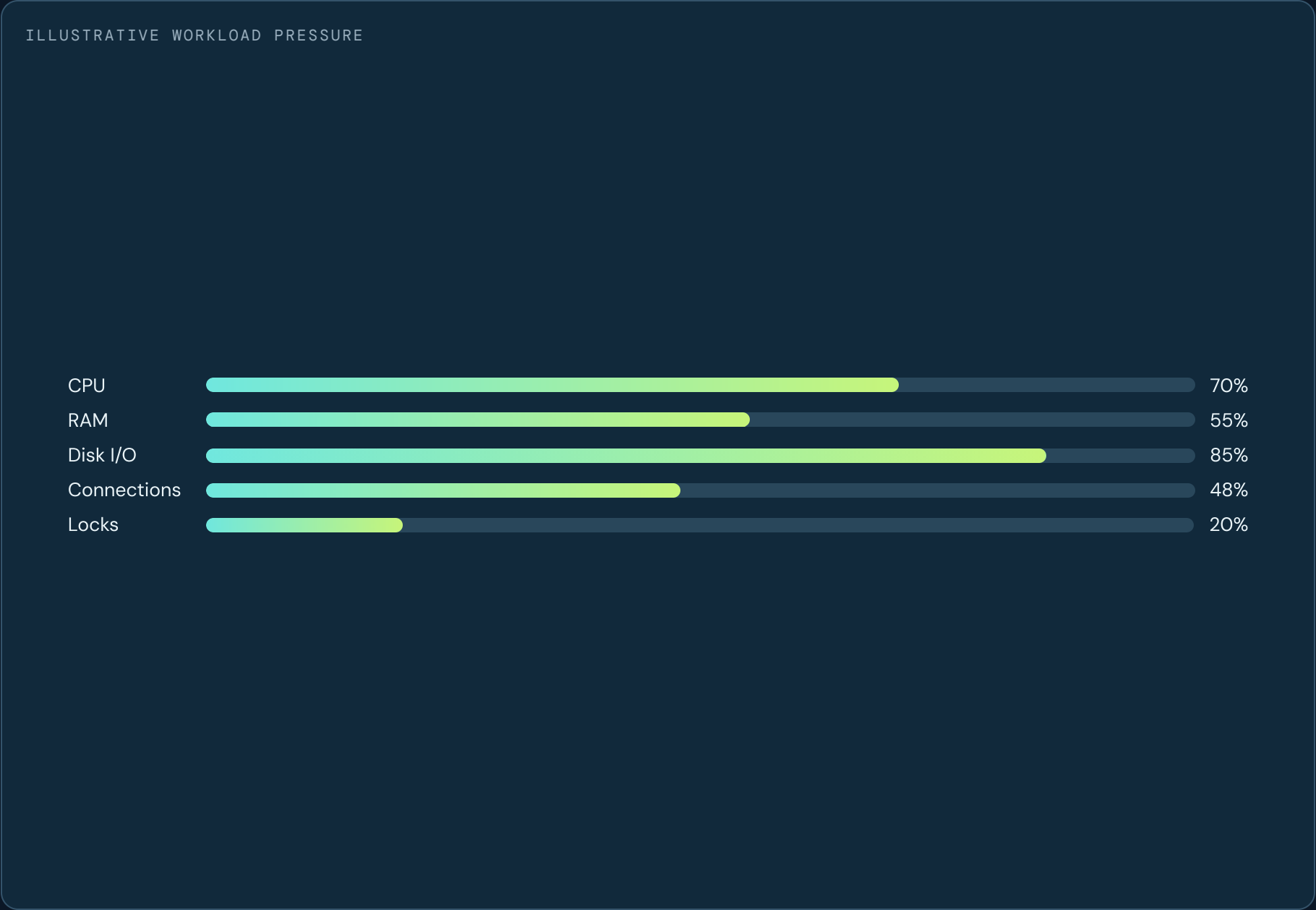
Technical detail

Chunk quality, metadata filters, hybrid search, ranking and citations affect answer quality. Retrieved text is untrusted content; the model must not treat it as system instructions.

```
SELECT content, embedding <=> $1::vector AS distance  
FROM chunks ORDER BY embedding <=> $1::vector LIMIT 3;
```

Scale the bottleneck you measure

Load balancing, stateless APIs, pools, cache, replicas, queues and observability each have a job.



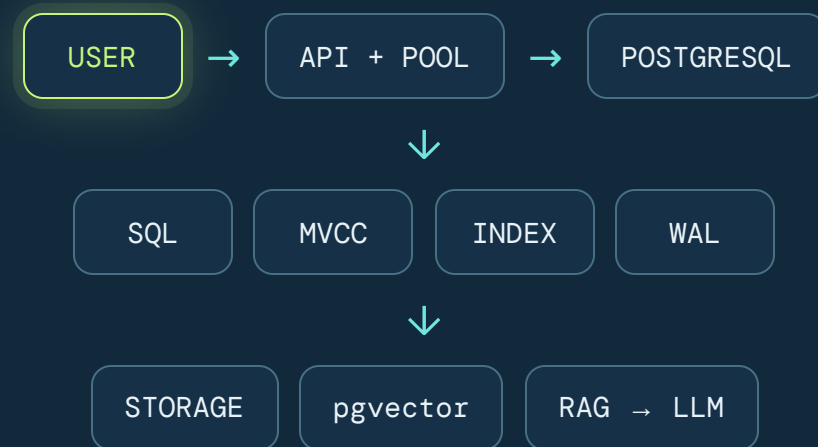
Technical detail

Throughput, p95 latency, CPU, I/O, locks, cache hit, queue wait and replication lag guide changes. Dashboard values here are illustrative.

```
SELECT * FROM pg_stat_activity;
```

One request. One database. Many guarantees.

SQL, transactions, plans, pages, WAL, maintenance, pooling and retrieval now form one picture.



Technical detail

Ten ideas: SQL is planned; transactions protect changes; MVCC versions rows; indexes trade read speed for write work; EXPLAIN reports execution; VACUUM maintains reuse; WAL enables recovery; pools bound concurrency; RLS limits rows; pgvector can support RAG.

-- Observe. Measure. Change one thing. Measure again.